Implementation of DiP and WS Accelerators on RISC-V Pipeline with BF16 on FPGA

***Abstract*—Weight-Stationary (WS) and Diagonal-Input Permutated (DiP) systolic dataflows have been compared analytically and on ASIC. However this has not been implemented on a real RISC-V SoC on FPGA silicon across both integer and floating-point precision. This paper closes that gap, we implement a 5-stage RV32I(F) RISC-V core coupled to a systolic-array accelerator via custom instructions, supporting both WS and DiP dataflows in integer and BF16 precision. The design has been synthesized and timing-closed across four FPGA families spanning three process generations: Virtex-7, Kintex-7, Zynq-7000, and Zynq UltraScale+. We derive exact cycle-accurate latency formulas directly from RTL — $3N^2 + 4N + 28(WS)$ and $3N^2 + 3N + 29(DiP)$ both for BF16 precision which is validated across array sizes. Beyond dataflow, we find the cost of BF16 precision is sharply metric-dependent. It is negligible in latency and increases a flat 3–20% in power. BF16 grows from 1.6× to nearly 5× the integer area footprint as N scales. The paper's central finding, is a bottleneck neither dataflow nor precision choice can fix: element-by-element memory streaming that scales by N^2, accounting for over 95% of total latency by $N = 32$. Thus, indicating future acceleration effort is better spent widening the memory interface than further optimizing the array itself.**

***Index Terms*—Systolic Array, Weight-Stationary Dataflow, Diagonal-Input Permutated Dataflow, RISC-V, Hardware Accelerator, FPGA, BFloat16, Fixed-Point Arithmetic, Memory Bandwidth**

I. Introduction

Matrix multiplication is extensively employed in scientific computing workloads and the attention/FFN layers of modern transformer models. Acceleration of matrix multiplication is crucial for high-performance computing, edge AI, and IoT [1], [2]. Matrix multiplication constitutes a bulk of the compute cost in modern AI, making energy-efficient methods to accelerate it essential. Among many approaches, systolic arrays have emerged as efficient due to rhythmic data movement and high throughput [3]. These architectures have been widely adopted in commercial accelerators such as Google's TPU [4], while alternative dataflows have also been explored, such as Eyeriss [5]. Conventional weight-stationary arrays use FIFO synchronization, introducing latency and silicon cost. Motivated by variants like Meissa [6], a novel method, DiP, was recently introduced [7]. This method eliminates FIFOs entirely, providing up to 50% higher throughput and 2× energy efficiency compared to conventional designs.

This paper explores the DiP and WS dataflows integrated into a RISC-V pipeline across various FPGAs and matrix sizes. We implement both integer (fixed-point) and custom floating-point (BF16) variants of each accelerator, enabling a fair comparison of dataflow and numerical precision. We compare the results against the weight-stationary dataflow to establish a baseline [7], measuring speedup, resource utilization, and power trade-offs introduced by each design.

Edge-AI applications require efficient power utilization and computation. We compare performance across various FPGA boards, evaluating the efficiency of each accelerator across different matrix sizes to maximize throughput and energy efficiency for low-power AI applications. Moreover, coupling the accelerator with the RISC-V pipeline enables a compact SoC that can be deployed for both matrix acceleration and general CPU processes.

The hardware accelerator is embedded within a modified RISC-V core [8], [9], and is activated through a custom instruction. Upon receiving this instruction, the core delegates the matrix multiplication task to the accelerator and stalls until completion. The accelerator passes the data from memory into its arrays through an FSM, which captures the result and writes it back into data memory during the write-back stage.

The contributions of this paper are as follows:

- A 5-stage pipelined 32-bit RISC-V processor integrated with multiple processing elements for parallel matrix multiplication has been developed.
- DiP and WS-based matrix multiplication accelerators, implemented in both integer (fixed-point) and custom floating-point (BF16) variants, along with the required FSM to coordinate the accelerator with the RISC-V pipeline.
- A precision data flow and tradeoff study comparing integer and floating-point implementations across both WS and DiP, quantifying resource utilization, latency, and power differences.
- All proposed modules are developed, written in Verilog, synthesized, and implemented on an FPGA platform through the Xilinx Vivado design suite.

The rest of the paper is structured as follows: Section II discusses the relevant literature and background for accelerators in DiP, WS, and their optimization. Section III introduces the methodology for implementing DiP and WS on RISC-V cores. Section IV discusses the results and analysis.

II. Literature Review

Matrix multiplication is used in modern applications, including classical high-performance computing (HPC) and deep learning workloads. In HPC, speeding up matrix multiplication has been at the forefront for a long time due to its large latency in the pipeline [1]. Deep learning relies heavily on transformers and attention mechanisms, which use large-scale

matrix multiplication. For these reasons, accelerating matrix multiplication is a necessity. Its importance extends further to NLP and vision tasks.

Systolic arrays were first introduced by Kung in 1982 [3]. He proposed small compute elements—"Processing Elements (PEs)"—arranged in a grid format to parallelize computation. The rise of AI has led this architecture to be refined over multiple iterations. Some approaches include weight-stationary, input-stationary, row-stationary, and output-stationary. Google TPUs adopt the weight-stationary dataflow [4], while Eyeriss adopts a row-stationary approach optimized for memory bottlenecks [5]. Conventional weight-stationary arrays require FIFO synchronization, introducing latency, routing congestion, and silicon overhead. The paper on Meissa [6] attempted to tackle this by separating multipliers and adders but faced scalability limits due to bulky adder trees. Recent developments introduced a novel approach to this problem in the form of Diagonal-Input Permutation (DiP). This method achieves up to 50% higher throughput and 2× energy efficiency compared to conventional WS designs [7].

Integrating these accelerators with the RISC-V flow provides a flexible testbed for applicability. RISC-V's open ISA supports custom instructions, allowing for application-specific acceleration. Previous work has demonstrated LU decomposition accelerators [9], sparse matrix extensions [10], and ISA extensions for real-time systems [11]. The SUMMA accelerator relies on a centralized controller and broadcasting models to feed PEs [12]. In contrast, our architecture relies on true PE-to-PE propagation, highlighting the novelty of integrating DiP and WS flows within a RISC-V pipeline along with floating-point support.

Beyond dataflow, numerical precision presents an independent trade-off in FPGA-based accelerator design. Fixed-point arithmetic typically achieves lower logic utilization and higher clock speed than floating-point on FPGA [13], motivating quantized variants of established architectures—for example, a recent study compares FP16 and quantized INT16 variants of a systolic array accelerator on FPGA in terms of energy and accuracy [14]. For the floating-point side, the bfloat16 (BF16) format has become a common choice in AI accelerators due to its FP32-matching dynamic range and compact 8-bit mantissa [15]. While [14] evaluates a precision trade-off on a single dataflow, no existing work jointly varies both dataflow (WS vs. DiP) and precision (integer vs. BF16) within a RISC-V-coupled accelerator.

III. METHODOLOGY

This section is split into seven parts: Part A covers the system overview. Part B explains the overall flow and the decisions made to integrate the accelerator into the RISC-V pipeline. Part C describes the dataflow routing used in DiP and WS arrays. Part D details the fundamental compute block (the processing element) and the architectural choices for integer and floating-point logic. Part E discusses the algorithms used for fixed-point accumulation. Part F covers the interaction with the processor, including the firmware and custom assembler. Finally, Part G outlines the FPGA implementation constraints and target platforms.

A. System Overview

The proposed architecture integrates the matrix multiplication accelerator with a 32-bit RISC-V processor. A custom instruction set diverts computation to the accelerator, which utilizes a unified shared memory subsystem for operand fetching and result write-back. The processing elements and systolic arrays operate in either a native integer (fixed-point) dataflow or in Brain Float 16 (BF16) with fixed 32-bit accumulation for maximum accuracy but reduced compute overhead.

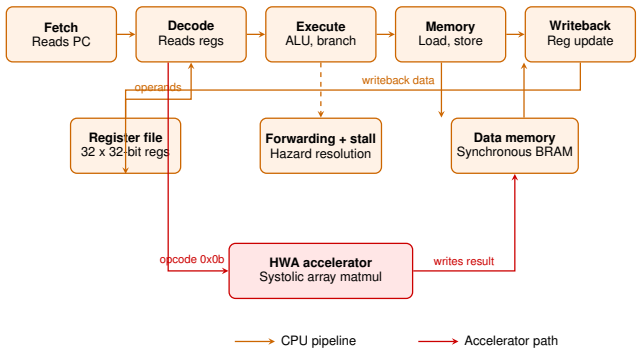


Fig. 1: RISC-V Pipeline Integration. Standard operations route through the ALU, while custom matrix instructions trigger the accelerator, pausing the CPU until the parallel computation is complete.

As shown in Figure 2, the centralized controller takes over memory access, streaming data directly into the local buffers of the systolic array. The computation is triggered automatically upon loading the final block of operands, ensuring a seamless, pipelined data flow without dead cycles.

B. RISC-V Processor Integration

The processor contains 5 stages in the pipeline – Instruction Fetch (IF), Instruction Decode (ID), Execution (EX), Memory Access (MEM), and Write Back (WB). The processor implements RV32I base architecture with an integrated RV32F extension to perform floating-point arithmetic operations. To activate the accelerator a custom opcode `7'b0001011` is implemented. the control unit identifies this opcode and directs the 3-bit `funct3` field directly to the HWA to dictate the specific matrix operation (`Load-A`, `Load-B and Compute`, or `Store-C`). The HWA asserts a stalling signal that freezes the processor's Fetch and Decode stages. This prevents hazards while the CPU waits for multiple clock cycles while the HWA finishes its job. The `Load-B and Compute` instruction streams the final operands and automatically triggers the parallel array computation. When calculations are finished, the HWA executes the `Store-C` command and temporarily taking control of the memory write-back bus to stream the computed matrix results from its local buffers back into the system's main data memory. After this it releases the pipeline from the stall, allowing processes to resume normally.

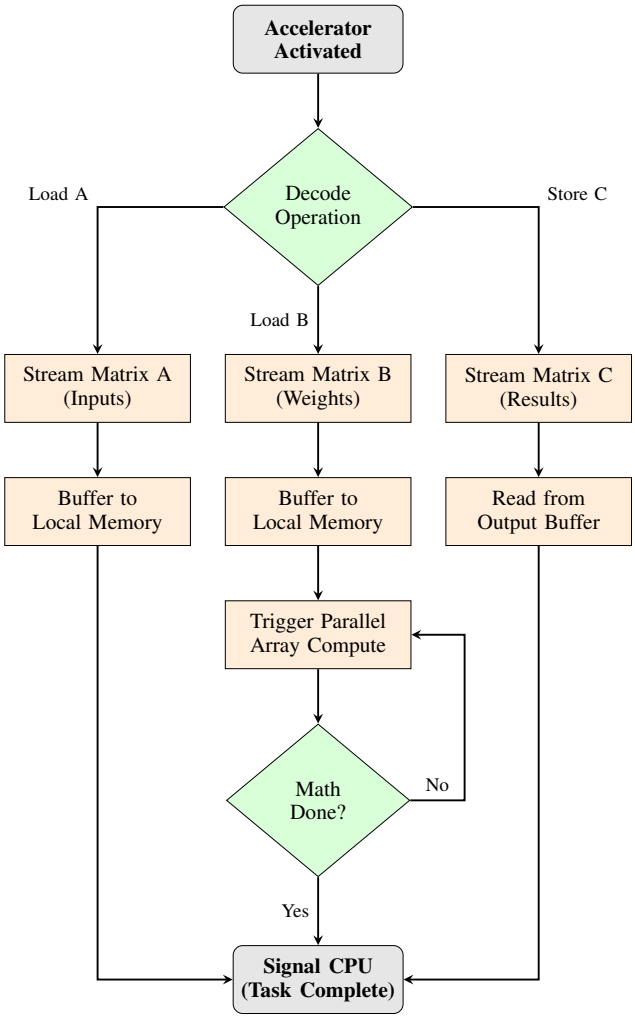


Fig. 2: Internal flow of the Hardware Accelerator. The controller decodes the specific matrix task and streams the data directly between the main memory and the localized systolic array buffers.

C. Systolic Array Dataflows: WS and DiP

The hardware accelerators consist of an $N \times N$ grid of PEs. Data movement differs fundamentally between the two setups. In the WS setup, the weight matrix is loaded into the PEs and remains stationary while the activation matrix flows horizontally through the grid. Synchronization is achieved using deep row-skew input FIFOs, which introduce latency and silicon overhead.

The DiP structure approaches this differently. To support diagonal routing without scrambling the mathematics, the weight matrixis permuted on the fly by the FSM and is streamed from local BRAM banks. This ensures operands collide correctly within the array. Activations then move diagonally by interconnecting the PEs.

To accommodate floating-point operations, some techniques had to be used. The entire pipeline operates in a BF16 format. To prevent constant normalization after the multiplication operation in each PE, the partial sum is allowed to accumulate in a fixed-32 format, only finally normalizing after the entire partial sum has been calculated. This saves on LUT cost and increases speed and accuracy. This boundary conversion overhead varies by architecture: it requires 4 cycles in the WS array (3 cycles more than the integer version) and 3 cycles in the DiP array (2 cycles more than the integer version). This normalization happens outside the PE grid once the partial sums are fully calculated, thus it does not need to be implemented in every PE. The integer version does not have this process, leading to lower LUT and power consumption.

D. Processing Element Architecture — Integer and BF16 Variants

The integer version of the PE multiplies 16-bit numbers to produce 32-bit results, which are accumulated in 32-bit registers.

The floating-point version is more complex. BF16 numbers are multiplied to create a 32-bit floating-point product. Instead of normalizing back to 16 bits, the module extracts the sign, exponent, and mantissa. The exponent determines a dynamic shift amount, which is applied to the mantissa to align the floating-point value into an integer representation. After applying the sign bit, this converted integer is added to the incoming partial sum using an integer adder, which are accumulated in 32-bit registers.

E. Fixed-Point Accumulation and Precision Conversion

To save power and silicon area, floating-point adders are not instantiated in every PE. The PE performs multiplication in the BF16 domain, but accumulation occurs in a fixed 32-bit format. This eliminates the need for continuous exponent matching and normalization within the replicated $N \times N$ grid.

Algorithm 1: BF16 PE Multiply–Accumulate with Deferred Conversion

Input: BF16 operands a, b; running accumulator acc_{fx} (32-bit)
Output: Updated fixed-point accumulator acc_{fx}

$prod_{fp} \leftarrow \text{BF16Multiply}(a, b)$
$prod_{fx} \leftarrow \text{BF16ProdToFixed}(prod_{fp})$ `// Cast to fixed-point`
$acc_{fx} \leftarrow acc_{fx} + prod_{fx}$ `// No per-PE rounding`
return acc_{fx}

Before writing results to memory, values are converted back into BF16 format. This boundary conversion involves leading-zero detection, mantissa normalization, and round-to-nearest-even logic. To avoid bottlenecks, the converter is mapped into a dedicated pipeline at the periphery of the array (4 cycles for WS, 3 cycles for DiP), reducing per-PE overhead.

Algorithm 2: Fixed-Point-to-BF16: Boundary Output Conversion

Input: 32-bit fixed-point accumulator value acc_{fx}
Output: BF16 result r

$s \leftarrow \text{Sign}(acc_{fx})$
$mag \leftarrow |acc_{fx}|$
$msb \leftarrow \text{FindHighestSetBit}(mag)$ `// Leading zero detect`
$e \leftarrow msb - offset_{fx} + bias$
$m \leftarrow \text{ExtractTopMantissaBits}(mag, msb, 7)$
if $needs_rounding$ **then**
  $(e, m) \leftarrow \text{RoundAndRenormalize}(e, m)$
$r \leftarrow \text{Pack}(s, e, m)$
return r

IV. Experimental Results

A. Latency

1) Latency: Isolated from memory transfer overhead, the hardware compute cycles for the array are:

$$Compute_{DiP}(N) = 3N + C$$

$$Compute_{ws}(N) = 4N + C$$

The constant C represents the fixed FSM coordination overhead (WS integer $C = 19$, DiP integer $C = 21$, WS BF16 $C = 28$, DiP BF16 $C = 29$).

Total measured pipeline latency, however, is dominated by a $3N^2$ memory transfer penalty (N^2 cycles each to load matrices A and B, and store C). Because the systolic array naturally reuses data, this $3N^2$ overhead applies only to the initial operation. For continuous workloads, Matrix A remains stationary in local buffers, meaning subsequent operations only require N^2 cycles to load the next Matrix B and store C.

Currently, the top-level FSM executes these memory transfers and computations sequentially due to the RISC-V SoC's shared single-port memory bus. While the FPGA silicon contains native dual-port BRAMs, exploiting them would require upgrading the system to a dual-bus memory controller. Such an upgrade would allow Matrix B to stream into local memory while the previous matrix computes, effectively hiding the N^2 load latency and allowing the accelerator to continuously operate at its true $3N$ or $4N$ hardware limit.

B. Area

Both integer and BF16 versions scale roughly N^2. DiP initially consumes more LUT's compared to WS (for $N \leq 4$), but this trend changes and by $N = 25$ it approaches the paper's ASIC predicted range of 5-8% on the FPGA also.The efficiency of implementation varies from board to board, as shown in Table I and Fig. 4.

Board	N=2	N=25
Virtex-7	1.59x	4.68x
Kintex-7	1.74x	5.29x
Zynq UltraScale+	1.62x	5.34x
ZedBoard (to N=12)	1.62x	4.01x (at N=12)

TABLE I: LUT cost ratio (BF16 ÷ Integer) for WS at N=2,25

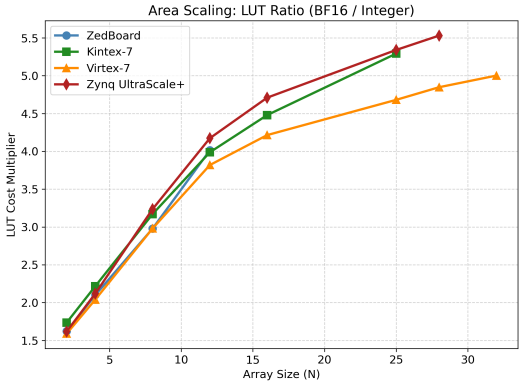


Fig. 4: Area Scaling: LUT Ratio (BF16 / Integer) across different boards.

C. Power

BF16 consumes from 3% - 20% more power across all boards as the matrix size increases. Integer and BF16 consume similar power causing the BF16/integer graph to remain flat across N. This can be explained as static and clock-distribution power dominate total on-chip power draw at every scale. Switching logic doesn't affect power too much, thus even a 5x increase in logic footprint barely moves the total.

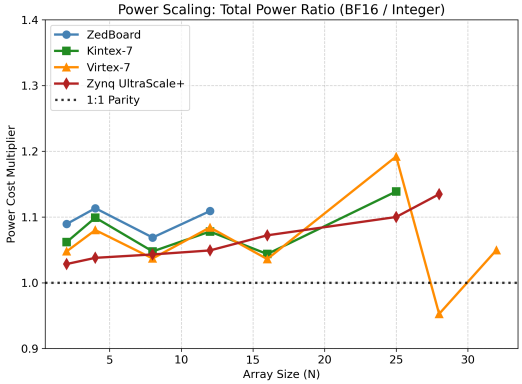


Fig. 5: Power Scaling: Total Power Ratio (BF16 / Integer) demonstrating the flat power penalty.

D. Timing (WNS)

Timing margin impact of BF16 precision is board-dependent rather than uniform. ZedBoard shows a dramatic collapse in margin, while the remaining three boards show inconsistent or reversed patterns. Kintex follows a non-monotonic sequence,

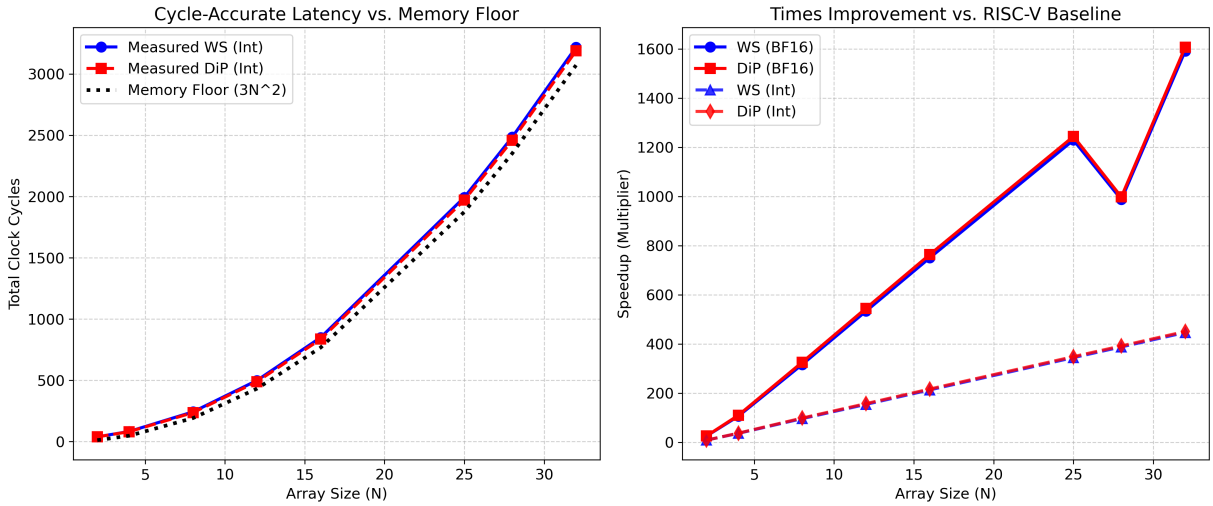


Fig. 3: Cycle-Accurate Latency vs. Memory Floor and Times Improvement vs. RISC-V (BF16).

while the Zynq UltraScale+ favoring BF16 until $N = 28$. The Virtex shows smooth but partial decline. Hold-time margins (WHS) remained positive throughout the sweep but narrow (30–44 ps across configurations). All designs met DRC and route-status compliance cleanly, with zero violations. Table II highlights the exact minimum and maximum WNS extremes recorded for each board across all tested matrix sizes.

TABLE II: Worst Negative Slack (WNS) Extremes by Board across all Array Sizes

Board	Precision	Min WNS (ns)	Max WNS (ns)
ZedBoard	Integer	2.719	3.723
	BF16	0.237	1.641
Kintex-7	Integer	2.263	3.634
	BF16	2.061	5.434
Virtex-7	Integer	3.397	6.028
	BF16	1.684	5.455
Zynq UltraScale+	Integer	5.647	7.397
	BF16	4.695	8.341

E. Accuracy

The numbers used in testing were floating point numbers $\in (-2, 2)$. This makes it especially difficult for RISC-V pathway to maintain high accuracy due to its element by element multiplication and normalization. Accelerators perform exceptionally in this is reflected in the mean absolute percentage deviation (MAPD) where HWA stays flat and low (0.42–1.70%) from $N = 2$ to $N = 32$, while the scalar RISC-V reference's error climbs sharply across the same range (0.70% $\rightarrow$ 16.21%).

The integer implementations show a less dramatic speedup multiplier than their BF16 counterparts. However, this is because the RISC-V software baseline for integer math is natively faster, whereas the scalar BF16 baseline relies on slow software emulation in C, artificially inflating the apparent hardware speedup for BF16.

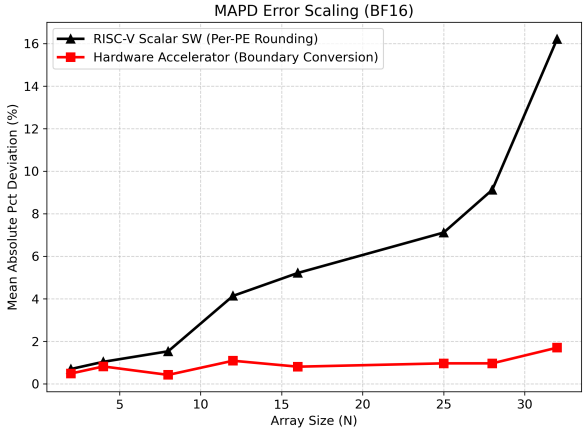


Fig. 6: MAPD Error Scaling (BF16): Hardware boundary conversion vs RISC-V per-PE rounding.

V. Conclusion

The paper set out to determine whether the DiP dataflow's advantage over WS, established analytically and on ASIC in prior work, is maintained once both are implemented within a full RISC-V SoC on real FPGA silicon across both integer and BF16 precision. The cycle-accurate advantage holds across all four FPGA boards. The pipeline introduces a memory bottleneck that scales as N^2, accounting for almost 96% of the total latency by $N = 32$. Between precisions, the cost of BF16 support is highly asymmetric across metrics: negligible in latency (a small constant-cycle overhead) and flat in power consumption, but substantial and growing in area, increasing from 1.6$\times$ at $N = 2$ to roughly 5$\times$ at $N = 32$. Implementation across four board families revealed

board-dependent timing closure costs for BF16 that are most severe on the resource-constrained ZedBoard and less severe or inconsistent on newer boards. Future work should target the now-dominant memory bottleneck directly, as dataflow optimization alone cannot overcome a cost that dictates the vast majority of latency at scale.

REFERENCES

[1] M. Schatz et al., "Parallel matrix multiplication," *High Performance Computing*, 2016.

[2] A. Vaswani et al., "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.

[3] H. T. Kung, "Why systolic architectures?" *Computer*, vol. 15, no. 1, pp. 37–46, 1982.

[4] N. P. Jouppi et al., "In-datacenter performance analysis of a tensor processing unit," in *Proceedings of the 44th annual international symposium on computer architecture*, 2017, pp. 1–12.

[5] Y. Chen et al., "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE journal of solid-state circuits*, vol. 52, no. 1, pp. 127–138, 2016.

[6] M. Asgari et al., "Meissa: A fast and scalable systolic array," 2020.

[7] A. Abdelmaksoud et al., "DiP: Diagonal-Input Permutation for FIFO-Free Systolic Arrays," 2024.

[8] Author Name, "Optimized SUMMA Accelerator on RISC-V for FPGA-Driven Parallel Matrix Multiplication," *IEEE*, 2024.

[9] B. Bhargavi et al., "Optimizing LU Decomposition on RISC-V," 2024.

[10] T. Titopoulos et al., "IndexMAC: Sparse Matrix Extensions," 2024.

[11] C. Cui et al., "RISC-V ISA extensions for real-time systems," 2023.

[12] R. A. Van de Geijn et al., "SUMMA: Scalable universal matrix multiplication algorithm," *Concurrency: Practice and Experience*, vol. 9, no. 4, pp. 255–274, 1997.

[13] D. L. N. Hettiarachchi, V. S. P. Davuluru, and E. J. Balster, "Integer vs. floating-point processing on modern FPGA technology," *10th Annual Computing and Communication Workshop and Conference (CCWC)*, 2020, pp. 0059–0065.

[14] A. Sanchez et al., "Energy and Precision Evaluation of a Systolic Array Accelerator Using a Quantization Approach for Edge Computing," *Electronics*, vol. 13, no. 14, p. 2822, 2024.

[15] D. Kalamkar et al., "A Study of BFLOAT16 for Deep Learning Training," *arXiv preprint arXiv:1905.12322*, 2019.